u3lib: An AI-Assisted Computational Framework for U(3) Symmetry Algebra, Wigner Coupling, and Racah Recoupling Coefficients

Phong Dang,[1] Daniel Langr,[2] Tomas Dytrych,[3,1] Feng Pan,[4,1] and Michela Negro[1]

[1]*Department of Physics and Astronomy, Louisiana State University, Baton Rouge, LA 70803, USA*
[2]*Department of Computer Systems, Faculty of Information Technology, Czech Technical University in Prague, Prague, Czech Republic*
[3]*Nuclear Physics Institute, Czech Academy of Sciences, 250 68 Řež, Czech Republic*
[4]*Department of Physics, Liaoning Normal University, Dalian 116029, China*

u3lib is an AI-assisted C++ framework for U(3)/SU(3) representation-theory computations in nuclear structure and related applications. Besides the fast and accurate evaluation of U(3) Wigner coupling and Racah recoupling coefficients – implemented with the procedure of Dang *et al.* [1], which avoids the binomial coefficients and alternating sums of earlier formulations – the library exposes the full algebraic toolkit around those coefficients: irreducible representations (irreps) and their dimensions, Casimir invariants and conjugation; the canonical Gelfand–Tsetlin basis and the physical SU(3) ⊃ SO(3) basis, with their inner and outer multiplicities; the matrix elements of the U(3) generators on canonical states; the reduced matrix elements of the physical generators **L** and **Q**; the tensor-product decomposition; and the transformation between the physical and canonical bases. The C++ API is mirrored one-to-one by Python bindings, and a repository-local catalog of agent skills lets a code-executing AI assistant drive any of these capabilities from a natural-language request. This document is the reference manual for the public release.

PROGRAM SUMMARY

Program title: u3lib
Licensing provisions: open-source (to be finalized; MIT/Apache-class).
Code repository: https://github.com/ORG-TBD/u3lib (placeholder — the public repository URL will be finalized on release).
Programming language: C++ (C++20), with Python bindings via pybind11.
External libraries: Eigen3; Boost (headers); OpenMP; fmt (header-only, for formatted output); the libwignernj SU(2) Wigner $3j/6j/9j$ backend; and a null-space solver backend — the sparse solver `sns` (default), which falls back to a dense Eigen3 QR. The Python extension additionally uses pybind11.
Nature of problem: Fast, accurate, and reproducible evaluation of the Wigner coupling coefficients for the two group–subgroup chains SU(3) ⊃ SO(3) and U(3) ⊃ U(2) ⊃ U(1), the U(3) Racah recoupling coefficients (U-, Z-, and 9-($\lambda\mu$) coefficients), and the associated representation-theoretic quantities (dimensions, Casimirs, multiplicities, basis enumeration, generator matrix elements, and basis transformations) needed by symmetry-adapted nuclear-structure calculations.
Solution method: The coupling and recoupling coefficients are built from the highest-weight canonical reduced coefficients, obtained as the orthonormal null space of the linear system enforced by the U(3) raising generators, and propagated by the recursion relations of Ref. [1]; the physical SO(3)-coupled coefficients follow from the physical → canonical basis transformation.

I. INTRODUCTION

u3lib evaluates the coupling and recoupling coefficients of the unitary group U(3) and its special unitary subgroup SU(3), together with the representation theory on which they rest. SU(3) symmetry underlies the Elliott model and the symmetry-adapted no-core shell model, where such coefficients are consumed in large numbers; speed, accuracy, and a stable, documented set of phase and normalization conventions are therefore essential.

The coefficients are computed with the procedure introduced in Ref. [1], which replaces the binomial coefficients and alternating sums of classical approaches by the solution of a sparse linear (null-space) problem followed by recursion, improving both numerical stability and range. The mathematical formulation, definitions, and governing equations are given in that paper and are not reproduced here; this manual is a reference for the software. u3lib packages the engine as a reusable framework: in addition to the coefficients it provides irreps and their invariants, the canonical (Gelfand–Tsetlin) and physical (SO(3)-coupled) bases, the generator matrix elements, the tensor-product decomposition, and the physical → canonical transformation. The C++ API is header-only; Python bindings mirror it one-to-one; and an agent-skill catalog exposes every capability to an AI assistant. The remainder documents the

repository structure (Sec. II), the classes and interface (Sec. III), the Python bindings (Sec. IV), the agent-assisted interface (Sec. V), and the build (Sec. VI).

II. LIBRARY STRUCTURE

`u3lib` is a header-only C++ library with a thin build layer. The top-level directories are:

- `include/` — the library itself (all algorithms live here; user programs include these headers directly):
  - `U3irrep.h` — the irrep classes `su3::irrep`, `u3::irrep`;
  - `U3gelfand.h`, `internal/gelfand/` — the Gelfand state class `u3::gelfand`;
  - `U3basis.h` — basis enumeration (`gelfandBasis`, `physicalBasis`), the inner multiplicity `Kappa`, the `u3::physicalvector` class, and the generator-action predicates;
  - `U3decomposition.h` — the outer multiplicity `Rho` and the tensor-product `decomposition`;
  - `U3U2xU3U2.h`, `U3SO3xU3SO3.h` — the canonical and physical coupling coefficients and their databases;
  - `U3_U6Z6Coeffs.h`, `U3_U9Coeffs.h` — the U-, Z-, and 9-$(\lambda\mu)$ recoupling coefficients and their databases;
  - `U3SO3_to_U3U2U1.h` — the physical $\rightarrow$ canonical transformation;
  - `U3generatorsRME.h` — the physical-generator reduced matrix elements `RME_L`, `RME_Q`;
  - `Definitions.h` — the structured coefficient containers;
  - `u3lib.h` — the umbrella header and the `initialize`/`finalize` lifecycle.
- `src/` — small example driver programs (one `.cpp` per task) that call the headers and print results; compiled to `bin/`.
- `python/` — the pybind11 extension (`pyu3lib.cpp` $\rightarrow$ `u3lib.so`, built with `make python`) and a tutorial notebook (`tutorial.ipynb`).
- `agents/` — the AI-agent skill catalog: an index (`AGENTS.md`), shared runtime instructions, and one skill per capability.
- `test/` — the regression suite (`test_*.cpp`, the `*_cases.h` case sets, and `reference/` documented values), run by `make test`.
- `doc/` — this manual (LaTeX source).
- `bin/` — build output (not source).

III. CLASSES AND LIBRARY INTERFACE

This section documents every public class and its methods, and the database/container interface through which coefficients are reached. All types are in the namespace `u3` (SU(3)-only base members in `su3`). The algebraic types and their methods do not require initialization; the coefficient *databases* do. Table I is an at-a-glance map of the main entry points.

A. Lifecycle

In `u3lib.h`:

- `initialize()` / `initialize(&argc,&argv)` — set up the Wigner backend, the SO(3) CGC/Racah caches, and the U(3) coefficient databases;
- `finalize()` — tear everything down;
- `initialize_thread()` / `finalize_thread()` — per-thread setup for multithreaded use;
- `clear()` — drop cached coefficients while keeping the databases alive;

Header	Entry point	Task	Reference
`U3irrep.h`	`dim`, `C1/C2/C3`, `conjugate`	irrep dimension, Casimirs, conjugation	—
`U3basis.h`	`Kappa`, `gelfandBasis`, `physicalBasis`	inner multiplicity; basis enumeration	[1]
`U3decomposition.h`	`Rho`, `decomposition`	outer multiplicity; $A \otimes B$ series	Sec. IV.A [1]
`U3U2xU3U2.h`	`wigU3U2db`	U(3)⊃U(2) canonical reduced CG	Sec. IV.C–D [1]
`U3SO3xU3SO3.h`	`wigU3SO3db`	SU(3)⊃SO(3) physical CG	Eq. (40) [1]
`U3_U6Z6Coeffs.h`	`u3u6db`, `u3z6db`	U- and Z-recoupling (three irreps)	Eqs. (21),(22) [1]
`U3_U9Coeffs.h`	`u3u9db`	9-$(\lambda\mu)$ recoupling (four irreps)	Eq. (23) [1]
`U3SO3_to_U3U2U1.h`	`phys_to_cano`	physical → canonical transformation	Eqs. (35),(36) [1]
`U3generatorsRME.h`	`RME_L`, `RME_Q`	reduced matrix elements of **L**, **Q**	—

TABLE I: Main entry points of `u3lib`.

- `is_initialized()` — query the state.

Shared-backend lifecycle: if `u3lib` runs in the same process as another library that shares the Wigner backend, initialize all of them at startup and finalize only after all coefficient work is complete. Calling `finalize()` while another user of the backend is still active destroys the shared state and causes runtime errors.

B. Irreps: `su3::irrep` and `u3::irrep`

(`U3irrep.h`.) `su3::irrep` stores an SU(3) irrep; `u3::irrep` derives from it and adds the determinant row. **Constructors.**

- `su3::irrep()` — default (empty) SU(3) irrep.
- `su3::irrep(lambda,mu)` — SU(3) irrep from its labels.
- `u3::irrep()` — default (empty) U(3) irrep.
- `u3::irrep(n13,n23,n33)` — U(3) irrep from its three row labels.
- `u3::irrep(lambda,mu)` — normalized representative $[\lambda{+}\mu, \mu, 0]$.
- `u3::irrep(su3::irrep, N)` — fix the total quanta N.

Labels.

- `lam()` / `lambda()` — the SU(3) label λ.
- `mu()` — the SU(3) label μ.
- `n13()`, `n23()`, `n33()` — the three U(3) row labels (`u3::irrep`).

Dimension and invariants.

- `dim()` — the dimension (both classes).
- `C2()` — the quadratic SU(3) Casimir (both classes, shift-invariant).
- `C3()` — the cubic SU(3) Casimir (both classes, shift-invariant).
- `C1()` — the first-order U(1) Casimir = $n_{13}{+}n_{23}{+}n_{33}$ (total quanta), on `u3::irrep` only; `su3::irrep` has no `C1()` (SU(3) drops the U(1) trace generator).
- `quanta()` — synonym of `C1()`.

Other.

- `zero()` — test for the trivial irrep.
- `conjugate()` — the conjugate irrep (μ, λ).
- `conjugate_phase()` — the associated conjugation phase.
- `normalize()` — set $n_{33} = 0$ (`u3::irrep`).
- comparison operators and stream output.
- `writeBinary` / `readBinary` — binary serialization.

C. Gelfand states: `u3::gelfand`

(`U3gelfand.h`.) The canonical basis state $|n_{13}, n_{23}, n_{33}; n_{12}, n_{22}; n_{11}\rangle$. **Constructor.**

- `u3::gelfand(n13,n23,n33,n12,n22,n11)` — build a Gelfand state from its labels.

Labels and validity.

- `n13()` ... `n11()` — the six Gelfand-pattern labels.
- `check()` — test the betweenness condition.
- `normalize()` — shift $n_{33} \to 0$.

Subgroup labels.

- `lam()` / `lambda()` — the SU(3) label λ.
- `mu()` — the SU(3) label μ.
- `epsilon()` — the U(1) weight.
- `Lambdax2()` — $2\times$ the embedded SU(2) angular momentum.
- `MLambdax2()` — $2\times$ its projection.

Conjugation.

- `conjugate()` — the conjugate Gelfand state.
- `conjugate_phase()` — the associated conjugation phase.

Generator matrix elements. Each returns the matrix element of a U(3) generator E_{ij} acting on the state; a `_12`/`_22` suffix selects the target branch.

- `e11()`, `e22()`, `e33()` — the diagonal weight generators.
- `e12()` — the E_{12} element.
- `e21()` — the E_{21} element.
- `e23_12()`, `e23_22()` — the E_{23} element, per target branch.
- `e32_12()`, `e32_22()` — the E_{32} element, per target branch.
- `e13_12()`, `e13_22()` — the E_{13} element, per target branch.
- `e31_12()`, `e31_22()` — the E_{31} element, per target branch.

The raising-action predicates `E12`, `E23_12`, `E23_22`, `E13_12`, `E13_22` (in `U3basis.h`) identify the target state of each generator.

D. Physical states: `u3::physicalvector`

(`U3basis.h`.) The SU(3) ⊃ SO(3) reduced state $|(\lambda\mu)\,\kappa\,L\rangle$. **Constructor.**

- `u3::physicalvector(irrep, L)` — a physical state of angular momentum L.

Accessors.

- `irr()` — the underlying irrep.
- `lam()` / `lambda()` — the SU(3) label λ.
- `mu()` — the SU(3) label μ.
- `n13()`, `n23()`, `n33()` — the U(3) row labels.
- `L()` — the angular momentum.
- `kappa()` — the inner multiplicity of L (= `kappamax()`).

E. Free functions

Multiplicities.

- `u3::Kappa(irrep,L)` — inner multiplicity $\kappa_{\max}$ (also `kmax`, `mult`).
- `u3::Rho(A,B,C)` — outer multiplicity $\rho_{\max}$ of C in $A \otimes B$ (`U3decomposition.h`).

Bases.

- `u3::gelfandBasis(irrep)` — all Gelfand states of the irrep.
- `u3::physicalBasis(irrep)` — all physical states with nonzero κ.
- `u3::findGelfand(g,gArr)` — the index of a state within `gArr`.

Decomposition.

- `u3::decomposition(A,B)` — the full Kronecker series, returning (list of C, list of $\rho_{\max}$) with $\sum_C \rho_{\max}(C) \dim C = \dim A \dim B$.

Generator RMEs (`U3generatorsRME.h`). Each takes two `physicalvector` states (bra, ket) and two `std::vector<int>` out-parameters (which receive the 1-based multiplicity labels), and returns an `Eigen::MatrixXd` of shape $\kappa'_{\max} \times \kappa_{\max}$.

- `RME_L` — the angular-momentum RME, $\langle(\lambda\mu)\kappa' L'\|\mathbf{L}\|(\lambda\mu)\kappa L\rangle = \delta_{L'L}\sqrt{L(L{+}1)(2L{+}1)}$.
- `RME_Q` — the quadrupole RME, carrying an extra $\sqrt{2L'+1}$ (Sec. III H).

Transformation (`U3SO3_to_U3U2U1.h`). Both expand a physical state in the Gelfand basis, returning the canonical-state indices into `gArr=gelfandBasis(p.irr())` and an amplitude matrix with one orthonormal column per κ.

- `u3::phys_to_cano(p,M,gArr)` — the expansion at a single projection M.
- `u3::phys_to_cano_allM(p,gArr)` — the expansion over all M.

F. Coefficient containers

(`Definitions.h`.) Every database call returns one of these structured containers; indices are zero-based and bounded by the `*max()` counts. Accessors common to every container:

- `operator()(...)` — the coefficient at the given multiplicity indices (signature per container below).
- `data()` — the underlying flat buffer.
- `size()` — the number of stored coefficients.
- `empty()` — true for a forbidden or empty coupling.

The multiplicity-axis counts, one per line, and the `operator()` index order for each container:

`U3U2` — canonical coupling. Also exposes a flat `operator[]`.

- `rhomax()` — the outer multiplicity ρ.
- `operator()(rho)` — indexed by ρ.

`U3SO3` — physical coupling.

- `kappaAmax()` — inner multiplicity κ of leg A.
- `kappaBmax()` — inner multiplicity κ of leg B.
- `kappaCmax()` — inner multiplicity κ of leg C.
- `rhomax()` — the outer multiplicity ρ.
- `operator()(kappaA,kappaB,kappaC,rho)`.

`U3U6` — U-recoupling, $(AB)C \leftrightarrow A(BC)$.

- `rhoABmax()` — multiplicity of the $A \otimes B \to AB$ coupling.
- `rhoAB_Cmax()` — multiplicity of the $AB \otimes C$ coupling.
- `rhoBCmax()` — multiplicity of the $B \otimes C \to BC$ coupling.
- `rhoA_BCmax()` — multiplicity of the $A \otimes BC$ coupling.
- `operator()(rhoAB,rhoAB_C,rhoBC,rhoA_BC)`.

`U3Z6` — Z-recoupling, $(AB)C \leftrightarrow (AC)B$. It reuses the `U3U6` getter names for its second route:

- `rhoABmax()` — multiplicity of the $A \otimes B \to AB$ coupling.
- `rhoAB_Cmax()` — multiplicity of the $AB \otimes C$ coupling.
- `rhoBCmax()` — here the $A \otimes C \to AC$ channel.
- `rhoA_BCmax()` — here the $AC \otimes B$ channel.
- `operator()(rhoAB,rhoAB_C,rhoAC,rhoAC_B)`.

`U3U9` — 9-$(\lambda\mu)$ recoupling.

- `rhoACmax()` — multiplicity of the $A \otimes C \to AC$ coupling.
- `rhoBDmax()` — multiplicity of the $B \otimes D \to BD$ coupling.
- `rhoABCDmax()` — multiplicity of the $(AB)(CD) \to ABCD$ route.
- `rhoABmax()` — multiplicity of the $A \otimes B \to AB$ coupling.
- `rhoCDmax()` — multiplicity of the $C \otimes D \to CD$ coupling.
- `rhoACBDmax()` — multiplicity of the $(AC)(BD) \to ABCD$ route.
- `operator()(rhoAC,rhoBD,rhoABCD,rhoAB,rhoCD,rhoACBD)`.

In C++ the `U3U9` result is a view into a thread-local scratch buffer that the next database call overwrites; copy it before the next call. The other four own their buffers. (The Python bindings copy `U3U9` automatically.)

G. Coefficient databases

Reached through singletons; each returns a container above.

- `u3::wigU3U2db().get(` $n^a_{13}, n^a_{23}, n^a_{33}, n^a_{12}, n^a_{22}, \ldots^b, \ldots^c$ `)` → `U3U2` — U(3) ⊃ U(2) reduced coupling for $a \otimes b \to c$ (each state by its U(3) rows and U(2) sublabels n_{12}, n_{22}), indexed by ρ.
- `u3::wigU3SO3db().get_U3SO3(` $\lambda_1, \mu_1, L_1,\ \lambda_2, \mu_2, L_2,\ \lambda_3, \mu_3, L_3$ `)` → `U3SO3` — SU(3) ⊃ SO(3) reduced coupling, indexed by $(\kappa_A, \kappa_B, \kappa_C, \rho)$.
- `u3::u3u6db().get_U3U6(` $A, B, ABC, C, AB,\ \rho_{AB}, \rho_{AB_C},\ BC, \rho_{BC}, \rho_{A_BC}$ `)` → `U3U6` — recouples $(AB)C \leftrightarrow A(BC)$.
- `u3::u3z6db().get_U3Z6(` $B, A, ABC, C, AB,\ \rho_{AB}, \rho_{AB_C},\ AC, \rho_{AC}, \rho_{AC_B}$ `)` → `U3Z6` — recouples $(AB)C \leftrightarrow (AC)B$ (note the leading B, A).
- `u3::u3u9db().get_U3U9(` $A, B, C, D,\ AB, CD, AC, BD, ABCD,\ \rho_{AC}, \rho_{BD}, \rho_{ABCD},\ \rho_{AB}, \rho_{CD}, \rho_{ACBD}$ `)` → `U3U9`.

The multiplicity maxima are the `Rho` values for the corresponding channels, supplied in the fixed interleaved positions shown; a mismatch trips an internal assertion. In `get_U3U9`, note that the 3rd multiplicity argument ρ_{ABCD} corresponds to the $(AB)(CD) \to ABCD$ coupling route and the 6th (ρ_{ACBD}) to $(AC)(BD) \to ABCD$; swapping them trips an internal assertion. A forbidden or empty coupling returns an empty container (safe to query for shape, not to index).

A minimal C++ session:

```
u3::initialize(&argc, &argv);
auto blk = u3::wigU3SO3db().get_U3SO3(2,0,2, 2,0,2, 4,0,2);
double x = blk(0, 0, 0, 0);     // = -sqrt(2/9)
u3::finalize();
```

H. Conventions and normalization

Phase and normalization conventions follow Ref. [1]; documented values and the generation procedure are in `test/reference/`.

- **Multiplicity indexing.** Counts come from `Kappa`/`Rho`; container accessors use zero-based indices $0, \ldots, \max-1$. The generator-RME routines report 1-based labels in their out-parameters.
- **Canonical-coupling phase.** For each ρ block the phase and normalization are fixed by the highest-weight reduced coefficients – the orthonormal null space of the system imposed by E_{23} and E_{12} annihilating the highest-weight coupled state. For $\rho_{\max} > 1$ (and self-conjugate irreps) the solver fixes a valid orthonormal ρ basis (RREF + Gram–Schmidt + leading-sign); any alternative must be applied consistently to every ρ-carrying coefficient. Conjugation of a Gelfand state satisfies $C\,|g\rangle = (-1)^{n_{23}-n_{12}-n_{22}-n_{11}}\,|g.\text{conjugate}()\rangle$ (`gelfand::conjugate_phase()`).
- **Physical → canonical transformation.** A physical SO(3) state $|(\lambda\mu)\,\kappa\,L\,M\rangle$ is expanded in canonical Gelfand states $|g\rangle$ with $M = e_{11} - e_{22} = 2n_{11} - (n_{12} + n_{22})$. The highest-weight state $M = L$ is the null space of $L_+ = \sqrt{2}(E_{13} + E_{32})$; L_- fixes the lower-M phases within a κ multiplet. When compiled with `ADD_SU3LIB_PHASE_U3SO3`, distinct conjugate irreps obey $C\,|(\lambda\mu)\,\kappa\,L\,M\rangle = (-1)^{\lambda+\mu-L+M}\,|(\mu\lambda)\,\kappa\,L, -M\rangle$ (for $\lambda > \mu$ the transformation is built from the conjugate irrep with the phase factor $(-1)^{n_{22}+n_{11}-L}$). For a self-conjugate irrep (λ, λ) conjugation acts as an M-independent symmetric matrix D ($D^2 = \mathbb{1}$) that may mix κ.
- **Generator RME factor.** `RME_L`/`RME_Q` carry an extra $\sqrt{2L'+1}$, so $\langle \kappa L \| \mathbf{Q} \| \kappa' L' \rangle = (-1)^{L-L'} \langle \kappa' L' \| \mathbf{Q} \| \kappa L \rangle$.
- **Assertions are on.** Invalid labels or out-of-range indices abort in C++; the Python bindings guard database access and raise instead.

IV. PYTHON BINDINGS

The module `u3lib` mirrors the C++ API one-to-one; build it with `make python` and the interpreter whose ABI matches the extension. Irreps (`su3.irrep`, `irrep`), `gelfand`, `physicalvector`, the free functions `Rho`, `Kappa`, `decomposition`, `gelfandBasis`, `physicalBasis`, the physical-generator RMEs `RME_L`/`RME_Q`, the transformation `phys_to_cano`/`phys_to_cano_allM`, and the five database accessors are all exposed with the same names and semantics as Sec. III. The structured containers provide `__call__` (multiplicity-indexed), flat `data()` as a NumPy array, and a printing method; database access before `initialize()` or after `finalize()` raises `RuntimeError` rather than aborting. The extension links no MPI and uses `sns` or a dense QR for the null-space solve.

```
import u3lib as u3
u3.initialize()
try:
    blk = u3.wigU3SO3db().get_U3SO3(2,0,2, 2,0,2, 4,0,2)
    x = blk(0, 0, 0, 0)                 # (kappaA,kappaB,kappaC,rho) = -sqrt(2/9)
finally:
    u3.finalize()
```

A worked introduction covering irreps, bases, coupling, and recoupling is provided as a Jupyter notebook, `python/tutorial.ipynb`; build and toolchain details are in the top-level `README.md`.

V. AGENT-ASSISTED INTERFACE

`u3lib` ships a repository-local catalog of *agent skills* under `agents/`: an index (`agents/AGENTS.md`), shared runtime instructions, and one skill per capability. Each skill states its inputs, the exact library call, the convention, and a built-in numerical check. A code-executing AI assistant pointed at the catalog carries out the requested computation using the configured local library; no network service is involved.

The catalog also provides an `install` skill (`agents/install/SKILL.md`). On a fresh checkout the agent builds the library first — `make config` to resolve or download the dependencies, then `make` and `make test` — before running any calculation skill.

It is important to be precise about what the agent does and does not do. **The agent performs no physics and no mathematics itself.** It does not derive or approximate coupling coefficients, Casimirs, or matrix elements. Its role is *orchestration*: from a natural-language request it selects the matching skill, constructs valid inputs in the library's conventions, invokes the compiled C++/Python routine, and reports the returned numbers with their labels and the check that was run. All numerical work is done by the library; the agent is an interface that routes a request to the correct tool and formats the result, and it declines or asks for clarification when a request is ill-posed (for example, a request for the first-order Casimir of an SU(3) irrep, which is not defined).

Table II lists each skill with a representative prompt; the parenthetical results were checked against the library, so each prompt doubles as a self-test.

VI. BUILD AND INSTALLATION

`u3lib` is header-only; user programs include the headers in `include/` directly. The source will be hosted publicly at https://github.com/ORG-TBD/u3lib (placeholder, to be finalized on release). The build resolves dependency paths from a configuration environment rather than hardcoded paths; after sourcing it from the repository root the make targets are `make` (example programs → `bin/`), `make test` (rebuild and run the regression suite), and `make python` (the pybind11 extension). The regression tests check dimensions, Gelfand interlacing and ordering, SO(3) branching, canonical/physical coupling orthonormality across κ and ρ, conjugation identities, and documented analytic values; numerical agreement is tolerance-based rather than bitwise-identical across backends.

The external libraries are resolved by `make config` (which writes `config/config.generated.mk`); the two numerical libraries are either downloaded from their pinned upstream sources and built locally under `external-libs/` (not part of the repository), or taken from an existing install:

- **SU(2) Wigner backend:** libwignernj (https://github.com/susilehtola/libwignernj) — the sole backend, enabled via `USE_WIGNERNJ=1`, links dynamically (`-lwignernj`).
- **Null-space solver** (selected via `NULLSPACE_SOLVER=qr|sns`, default `sns`):
  - Dense QR — the always-available dense solver; requires no external sparse solver.

Skill	Example prompt (expected result)
`dim`	"Dimension of the SU(3) irrep $(2,1)$ – equivalently the U(3) irrep $[3,1,0]$ or any shift such as $[4,2,1]$?" (15; shift-invariant)
`casimirs`	"C_2, C_3 for the SU(3) irrep $(3,1)$, and C_1, C_2, C_3 for the U(3) irrep $[4,2,1]$." (SU(3): 50/3, 160/9; U(3): C_1=7, 32/3, 56/9 – C_1 is U(3)-only)
`cano-list`	"Enumerate the canonical Gelfand basis of $(2,0)$ with each state's $\epsilon, 2\Lambda, 2M$ labels." (6 states = dim)
`phys-list`	"List the physical SO(3) basis (κ, L) of $(4,2)$ and verify the dimension sum rule." (L=6, 5, 4, 3, 2, 0 with κ=1, 1, 2, 1, 2, 1; $\sum \kappa(2L{+}1)$=60)
`cano-generators`	"Act with E_{12} on the Gelfand state $\lvert 2,0,0;2,0;1\rangle$ of $(2,0)$; give the resulting state and matrix element." ($\sqrt{2}$)
`phys-generators`	"Compute $\langle (2,0)L{=}2 \Vert \mathbf{Q} \Vert (2,0)L{=}2\rangle$." ($\sqrt{35/3} \approx 3.416$)
`phys-to-cano`	"Express $\lvert (2,0)\,\kappa{=}1\,L{=}2\,M{=}0\rangle$ in the canonical Gelfand basis." (amplitudes $\sqrt{1/3}, \sqrt{2/3}$)
`inner-mult`	"SO(3) multiplicity κ of L=2 in $(4,2)$?" (2)
`outer-mult`	"Outer multiplicity ρ of $(1,1)$ in $(1,1)\otimes(1,1)$?" (2)
`decomposition`	"Full decomposition of $(2,0)\otimes(0,2)$ with outer multiplicities; check the dimension sum rule." ($(2,2){+}(1,1){+}(0,0)$)
`cano-coupling`	"U(3)$\supset$U(2) reduced coupling for the highest weights of $(2,0)\times(2,0)\to(4,0)$; verify it equals 1."
`phys-coupling`	"SO(3)-reduced coupling $(2,0)L{=}2 \times (2,0)L{=}2 \to (4,0)L{=}2$; check against $-\sqrt{2/9}$."
`u6recoupling`	"U-recoupling for $(1,0)\cdot(1,0)\cdot(1,0)$ through $(2,0)$ to $(3,0)$." (1)
`z6recoupling`	"Z-recoupling between the AB–C and AC–B routes for $(1,0)\cdot(1,0)\cdot(1,0)\to(3,0)$." (1)
`u9recoupling`	"9-$(\lambda\mu)$ recoupling for four fundamentals $(1,0)$ coupling to $(4,0)$." (1)
`conjugation`	"Conjugate the SU(3) irrep $(2,1)$ and its Gelfand states, reporting the phases." (irrep $\to (1,2)$, phase -1)

TABLE II: The `u3lib` agent skills with a representative prompt for each.

  - `sns` (https://gitlab.fit.cvut.cz/krastale/dp-sparse-nullspace) — sparse solver enabled via `USE_SNS=1`; builds with sparse representation (`-DNULLSPACE_SPARSE`) and falls back to dense QR for small or block-heavy cases.

- Eigen3 — https://eigen.tuxfamily.org
- Boost (headers) — https://www.boost.org
- OpenMP (via the compiler runtime, e.g. GCC's libgomp) — https://www.openmp.org
- fmt — https://github.com/fmtlib/fmt
- pybind11 (Python bindings) — https://github.com/pybind/pybind11

u3lib ships as an independent, self-contained repository with a uniform `make config`/`make`/`make test`/`make install` surface and no required environment variables. Its two numerical dependencies — the libwignernj Wigner backend and the `sns` null-space solver (dense-QR fallback) — are downloaded from their pinned upstream sources and built locally by `make config` (into `external-libs/`), or taken from an existing install. The heavier SLEPc/PETSc stack and the WIGXJPF backend used by earlier development builds have been dropped in favour of this lighter, portable footprint.

Acknowledgments

This library implements the U(3) coupling and recoupling procedure of Ref. [1]. The authors thank J. P. Draayer for his contributions to the underlying theory.

---

[1] P. Dang, J. P. Draayer, F. Pan, and K. S. Becker, "New Procedure for Evaluation of U(3) Coupling and Recoupling Coefficients," Eur. Phys. J. Plus **139**, 704 (2024).